🚀 Lecture 02.04: Keywords, Comments & AST Parsing

📌 Executive Architectural Summary

Python's grammar rules are built around reserved identifiers known as **Keywords**. These tokens have special meaning to the CPython tokenizer and Abstract Syntax Tree (AST) parser. Comments provide human documentation and are discarded during lexical analysis.

🔬 Core Engineering Mechanics

1. Python Keywords Lexicon

Python 3 features 35 reserved keywords (accessible via `keyword.kwlist`). These cannot be redefined:

- **Control Flow:** `if`, `elif`, `else`, `for`, `while`, `break`, `continue`, `return`, `pass`
- **Boolean & Logic:** `True`, `False`, `None`, `and`, `or`, `not`, `is`, `in`
- **Structure & Definitions:** `def`, `class`, `lambda`, `import`, `from`, `as`, `with`, `yield`
- **Exception Handling:** `try`, `except`, `finally`, `raise`, `assert`
- **Scope & Variables:** `global`, `nonlocal`, `del`
- **Asynchronous:** `async`, `await`

2. Comments vs Docstrings

- **Single-Line Comments (`#`):** Skipped by the tokenizer; zero bytecode is emitted.
- **Docstrings (`""" ... """`):** String literals placed as the first statement in a module, class, or function; retained at runtime in the `__doc__` attribute.

💻 Production Python Implementation

```
import keyword

# 1. Inspecting Reserved Keywords
print(f"Total Python Keywords: {len(keyword.kwlist)}")
print("Keywords:", keyword.kwlist[:10], "...")

# 2. Testing keyword validation
print("Is 'class' a keyword?", keyword.iskeyword("class"))
print("Is 'my_var' a keyword?", keyword.iskeyword("my_var"))
```

```
# 3. Demonstration of Docstrings
def compute_kinetic_energy(mass: float, velocity: float) -> float:
    """
    Computes Newtonian kinetic energy: E_k = 0.5 * m * v^2.
    """
    return 0.5 * mass * (velocity ** 2)

print("Function docstring:", compute_kinetic_energy.__doc__.strip())
```